• GitHub Dev Days I Perth

Secure by Design

For CLI Coding Agents

Practical security controls for CLI-based AI agents

George Coldham / Cloud Solution Architect AI Security @ Microsoft ANZ

2026

The Paradigm Shift

From Autocomplete to Autonomous

2020	2026
Code suggestions only	Read, write, execute

They are autonomous actors with full system access—not suggestion engines.

CLI Coding Agents in Practice

GitHub Copilot CLI

Local Terminal Agents

Claude Code

AI-powered dev tools with shell access

These are not edge cases.

These are already on developer machines.

Risk Assessment

The Risk Formula

$$\text{Risk} = C \times P \times T$$

C Capability

Agents can execute any shell command—read, write, install, push, fetch.

P Privilege

Environment grants access to filesystem, network, secrets, and git.

T Trust

Cannot distinguish user intent from malicious context in repos.

⚠ Controls must be enforceable, not advisory warnings

Architecture

How CLI Agents Work

Trust Boundary Challenge

The agent treats all context equally—it cannot distinguish between user's explicit instructions and malicious content hidden in repo files.

Architecture

Extensibility Expands the Attack Surface

- Plugins, MCP servers, and tools execute code
- Tool output becomes model context
- External systems influence agent decisions
- New credentials and API scopes introduced

Treat integrations like dependencies, not features

Attack Scenario

The Attack Flow

1

Plant

Attacker hides malicious instructions in README, issue comments, or code files

2

Trigger

Developer asks agent to work on repo; agent reads all files for context

3

Execute

Agent follows malicious context, executes commands as if from user

4

Impact

Secrets exfiltrated, backdoors installed, code compromised, lateral movement

☢ Blast radius scales with environment permissions—isolate before you delegate

Control Layer 01

Workspace Isolation

UNSAFE

```
$ claude ~/work/suspicious-repo
```

Running from home directory

- Full access to ~/.ssh
- All personal files exposed
- Git credentials accessible
- Ambient cloud credentials

Agent can access everything

CONTROLLED

```
$ claude /tmp/agent-workspaces/project
```

Running in isolated workspace

- No access to home directory
- Only project files visible
- Ephemeral, disposable
- Clean up on exit

Blast radius contained

Defense Architecture

Defense in Depth

06 **Agent Identity**
Dedicated bot account, signed commits, fine-grained tokens

05 **Command Gates**
Risk-based tiers: auto-allow, one-click, typed confirm

04 **Network Restrictions**
Default deny egress, allowlist only approved destinations

03 **Filesystem Boundaries**
Write allowlist, block sensitive paths, JIT secrets only

02 **Runtime Containment**
Devcontainers/Docker, no credential mounts, destroy after session

01 **Workspace Isolation**
Never run from home, use /tmp/agent-workspaces/, ephemeral

Each layer reduces blast radius. Together they create **defense in depth**.

Core Controls

Three Critical Controls

Non-negotiable security foundations

Isolated Workspace

Containerized environments with no access to host credentials or personal files

Just-in-Time Secrets

Short-lived tokens with explicit injection per command—never ambient credentials

Default Deny Egress

Block all outbound traffic by default, allowlist only approved registries and APIs

Build defenses that don't depend on user vigilance

Control Layer 05

Command Approval Flow

IF Command Type = ?

↓

LOW RISK

- npm run lint
- prettier
- git status

AUTO-ALLOW

MEDIUM RISK

- npm install
- pip install
- file writes

ONE-CLICK

HIGH RISK

- curl | bash
- rm -rf
- git push

TYPE "YES"

Risk-based tiers reduce friction while protecting dangerous operations. Log everything for audit.

Implementation

The 8-Point Baseline

1
Isolated
workspace/container

2
Write
boundaries (allowlist)

3
No ambient
secrets (JIT only)

4
Egress
controls (default deny)

5
Command
allowlist/gates

6
Scoped
git identity

7
Audit
logs

8
Required
CI checks

Build defenses that don't depend on user vigilance. Each layer reduces blast radius.

Transformation

Before & After

BEFORE

Ambient Credentials
AWS_PROFILE, KUBECONFIG in environment

Home Directory Execution
Full access to ~/.ssh, ~/.aws, personal files

Unrestricted Network
Any external host reachable, data exfiltration trivial

Shared Git Identity
Cannot distinguish human from AI commits

AFTER

Just-in-Time Tokens
15-min expiration, explicit injection per command

Isolated Workspaces
/tmp/agent-workspaces/, no credential access

Egress Allowlist
Default deny, npm/PyPI/GitHub/LLM only

Dedicated Bot Account
Signed commits, fine-grained tokens, 8h max

→ **Blast radius contained. Changes auditable.**

Boundaries Beat Warnings

Prompt warnings are advisory.

Technical controls are enforceable.

Build defenses that don't depend on user vigilance.

Questions?

https://my.awkward.space/

Thank you!

Secure by Design for CLI AI Coding Agents

Questions?